Predicting Song Popularity on Spotify Using Audio Features

Digital Assignment 1 (DA1) — Project Proposal

Field	Detail
Project Title	Predicting Song Popularity on Spotify Using Audio Features
Name	**[YOUR NAME]**
Registration Number	**[YOUR REGISTRATION NUMBER]**
Course	Predictive Analytics
Assessment	Digital Assignment 1 (DA1) — Project Proposal

2. Objective of the Proposed Work

Streaming platforms describe every track with numeric audio descriptors such as danceability, energy, valence and acousticness. This project asks whether the measurable character of a recording explains how popular it becomes on Spotify.

The objective is deliberately framed as a **variance-explained** question rather than an accuracy contest. Spotify's popularity score is a recency-weighted function of play counts, and play counts are driven by artist reputation, playlist placement and promotional spend, none of which is audible in the waveform. The scientifically meaningful target is therefore the **ceiling**: how far audio features actually go, and where the remaining explanatory power lives.

Research questions

- **RQ1** — How much variance in Spotify popularity is explained by audio features alone?
- **RQ2** — Which audio features carry that signal, and in which direction?
- **RQ3** — Does genre context materially improve on audio features alone?
- **RQ4** — Is popularity easier to predict as a binary outcome than a continuous score?

Expected outcome. A quantified answer to each question, supported by a reproducible modelling pipeline with explicit safeguards against data leakage, and an honest statement of the limits of audio-only prediction.

3. Models to be Applied and Justification

Models are introduced in increasing order of capacity so that each addition earns its complexity against the previous one. Both a regression formulation (continuous 0–100 score) and a classification formulation (binary popular / not popular) are applied, which is what allows RQ4 to be answered.

Model	Type	Justification
Mean predictor (DummyRegressor)	Baseline	Establishes the reference R^2 = 0. Without it, a weak model cannot be distinguished from a useless one. Every later result is measured against this.
Linear Regression (OLS)	Regression	Interpretable coefficients and a formal hypothesis-testing framework. Serves as the linear-signal benchmark: the gap between it and the tree models quantifies how much of the relationship is non-linear.
Ridge and Lasso	Regularised regression	Audio features are strongly collinear (VIF: tempo 15.3, energy 14.6, danceability 12.2). Regularisation stabilises coefficients under collinearity, and Lasso additionally performs embedded feature selection.

Model	Type	Justification
Random Forest	Ensemble (bagging)	Captures non-linear responses and feature interactions without manual specification, and is robust to the skewed feature distributions present in the data. Provides permutation-based importance for RQ2.
Gradient Boosting (HistGradientBoosting)	Ensemble (boosting)	Sequential error-correction typically outperforms bagging on tabular data; included as the strongest conventional competitor and as a check that Random Forest is not winning by accident.
Hurdle model (two-stage)	Specialised regression	About 10.5% of tracks sit at exactly zero popularity, and those zeros cluster by genre rather than scattering at random. Stage 1 classifies zero vs non-zero; stage 2 regresses on non-zero tracks only. This models the two data-generating processes separately instead of forcing one regressor to average across both.
Logistic Regression / RF / GB classifiers	Classification	The binary framing for RQ4, using the same three model families so the regression and classification comparison is like-for-like.

Justification for the overall modelling strategy

- **Two feature sets, every model.** Each model is fitted on audio features alone and then on audio + genre. The difference isolates the contribution of genre context, which is what RQ3 asks.
- **Artist-level features deliberately excluded.** Artist mean popularity is the strongest available predictor, but computing it across the dataset leaks the target. Excluding it caps accuracy and keeps the study faithful to its stated title.
- **All preprocessing inside a Pipeline**, so scalers and encoders are fitted per fold and never observe held-out data.

4. Datasets to be Used

Source. The Spotify Tracks Dataset — 114,000 tracks x 21 columns, BSD licensed, retrieved from its HuggingFace mirror (also mirrored on Kaggle). It contains track metadata, the eleven Spotify audio features, genre, and the target variable `popularity` (0–100).

> **Note on data provenance.** Spotify deprecated the `audio_features` and `audio_analysis` Web API endpoints on 27 November 2024; new applications now receive HTTP 403/410 and no official replacement exists. This data therefore cannot be collected first-hand, and the project necessarily uses an archived pre-deprecation snapshot. This constraint is disclosed rather than hidden, and is carried into the limitations of the analysis.

4.1 Separate datasets for training and testing

The source file is partitioned into three **physically separate, mutually disjoint** CSV files before any model is fitted. Partitioning is stratified on binned popularity so the target distribution — including the zero spike — is represented consistently in each.

Dataset file	Rows	Share	Role
`data/processed/train.csv`	62,704	70%	Model fitting and hyperparameter search (cross-validation is performed within this file only).
`data/processed/val.csv`	13,437	15%	Model selection and comparison between candidate models. Never used for fitting.
`data/processed/test.csv`	13,437	15%	Held-out final evaluation. Touched exactly once, after all modelling decisions are frozen.

4.2 Why deduplication precedes splitting

The raw file is constructed as 114 genres x 1,000 tracks, so a track belonging to several genres appears several times — up to nine. Of 114,000 rows, **24,259 (21.3%) are duplicate tracks**, leaving 89,741 unique `track_id` values. Splitting the raw file would place the **same song in both training and test sets**, inflating every reported score.

Deduplication is therefore applied first, and three assertions verify that no `track_id` appears in more than one partition. A further 163 physically impossible records (zero tempo, zero time signature, zero duration) are removed, giving a final analysis set of **89,578 unique tracks**.

4.3 Secondary dataset for external validation

To test whether findings generalise beyond a single scrape, the Spotify Hit Predictor Dataset (1960–2019, approximately 41,000 tracks with an independent hit/flop label) is proposed as an **external test set**. Because it originates from a different collection process and carries its own target definition, it provides a stronger generalisation check than an internal split alone. It also carries decade information, which the primary dataset lacks — see Section 8.4.

5. Metrics for Analysis and Justification

5.1 Regression metrics

Metric	Justification
R^2 (coefficient of determination)	The primary metric, because the research question is explicitly about variance explained. Directly interpretable as the proportion of variation in popularity accounted for by the model.
Adjusted R^2	The audio+genre feature set adds 114 one-hot genre columns. Adjusted R^2 penalises that added dimensionality, so the genre gain cannot be an artefact of simply having more predictors.
RMSE	Expressed in the units of the target (popularity points), which makes error magnitude concrete. Squared penalty makes it sensitive to the large errors produced by the zero-inflated mass.

Metric	Justification
MAE	Reported alongside RMSE precisely because it is not sensitive to those outliers. A large RMSE–MAE gap is itself diagnostic of the zero-inflation problem.

5.2 Classification metrics

Metric	Justification
ROC-AUC	Threshold-independent measure of ranking quality, so it evaluates the model without committing to a particular decision cut-off.
PR-AUC (average precision)	The positive class is a minority (about 23.7%). Precision-recall curves are the appropriate summary under imbalance, where ROC-AUC can appear optimistic. Always reported against the positive-rate baseline.
Precision, Recall, F1	Expose the operating trade-off that a single aggregate hides — a model may achieve high precision purely by predicting the positive class very rarely.
Confusion matrix	Reports the actual counts of each error type rather than a ratio.

Accuracy is deliberately excluded. With a 23.7% positive rate, a model that predicts “not popular” for every track achieves 76.3% accuracy while being entirely useless. Reporting accuracy here would be actively misleading, so it is replaced by PR-AUC measured against the positive-rate baseline.

5.3 Diagnostic and validity checks

- **Variance Inflation Factor (VIF)** — quantifies the multicollinearity that makes linear coefficients unreliable, and justifies the use of regularised models.
- **Permutation importance** — preferred over impurity-based importance for RQ2, because impurity measures systematically inflate correlated features.
- **SHAP values** — give per-feature direction and magnitude, not just rank ordering.
- **Shuffled-target check** — refitting on a permuted target must collapse R^2 to approximately zero. Any other result indicates leakage.

6. Tools and Libraries Used, with Justification

Tool / Library	Purpose	Justification
Python 3.11	Language	Standard environment for predictive analytics, with the mature scientific stack this project depends on.
pandas	Data manipulation	Efficient handling of the 114,000-row table; the grouped duplicate analysis and feature engineering are expressed directly in its API.
NumPy	Numerical computing	Vectorised transforms underlying the log and cyclic-encoding steps.
scikit-learn	Modelling and evaluation	Supplies every model family used, plus the `Pipeline` and `ColumnTransformer` abstractions that make leakage-free preprocessing structural rather than a matter of discipline.
statsmodels	Statistical diagnostics	Provides the variance inflation factor used for the formal multicollinearity test.
SHAP	Model interpretation	Game-theoretic attributions giving direction and magnitude per feature, needed for RQ2 where tree importances alone would be insufficient.
Matplotlib and seaborn	Visualisation	All figures are generated by code and regenerated from source, so no figure can drift from the data it depicts.
Jupyter Notebook	Analysis narrative	Interleaves code, output and reasoning, and is executed end to end to confirm reproducibility.
ReportLab	Document generation	Produces this PDF programmatically, so the document rebuilds automatically when results change.

Why scikit-learn rather than a deep-learning framework. The dataset is small, tabular and low-dimensional (roughly 15 predictors). Gradient-boosted trees and random forests are the established state of the art in this regime; a neural network would add substantial complexity and reduce interpretability without a corresponding gain, and interpretability is central to RQ2.

7. System Design: Flowchart and Module Diagram

Figure 1 traces the flow of data from the raw file through every processing function to the evaluation stage, annotated with the row count at each reduction. Figure 2 gives the module and class structure, listing the functions each module exposes and their inputs and return types. Both diagrams are generated from the actual source, so they cannot drift from the implementation.

7.1 Module responsibilities

Module	Inputs	Key functions	Outputs
`data_prep`	`dataset.csv`	`load_raw, deduplicate, clean, engineer, build, split, feature_columns`	`train.csv, val.csv, test.csv`; engineered feature matrix
`models`	feature column list; training partition	`make_preprocessor, regression_models, HurdleRegressor, evaluate, shuffle_check`	Fitted pipelines; metric dictionaries
`run_*` drivers	Prepared partitions	Experiment orchestration for RQ1-RQ4	`results_*.csv`; figures

Figure 1 — System Flowchart: Data Pipeline and Modelling Stage

Function names correspond exactly to src/data_prep.py and src/models.py

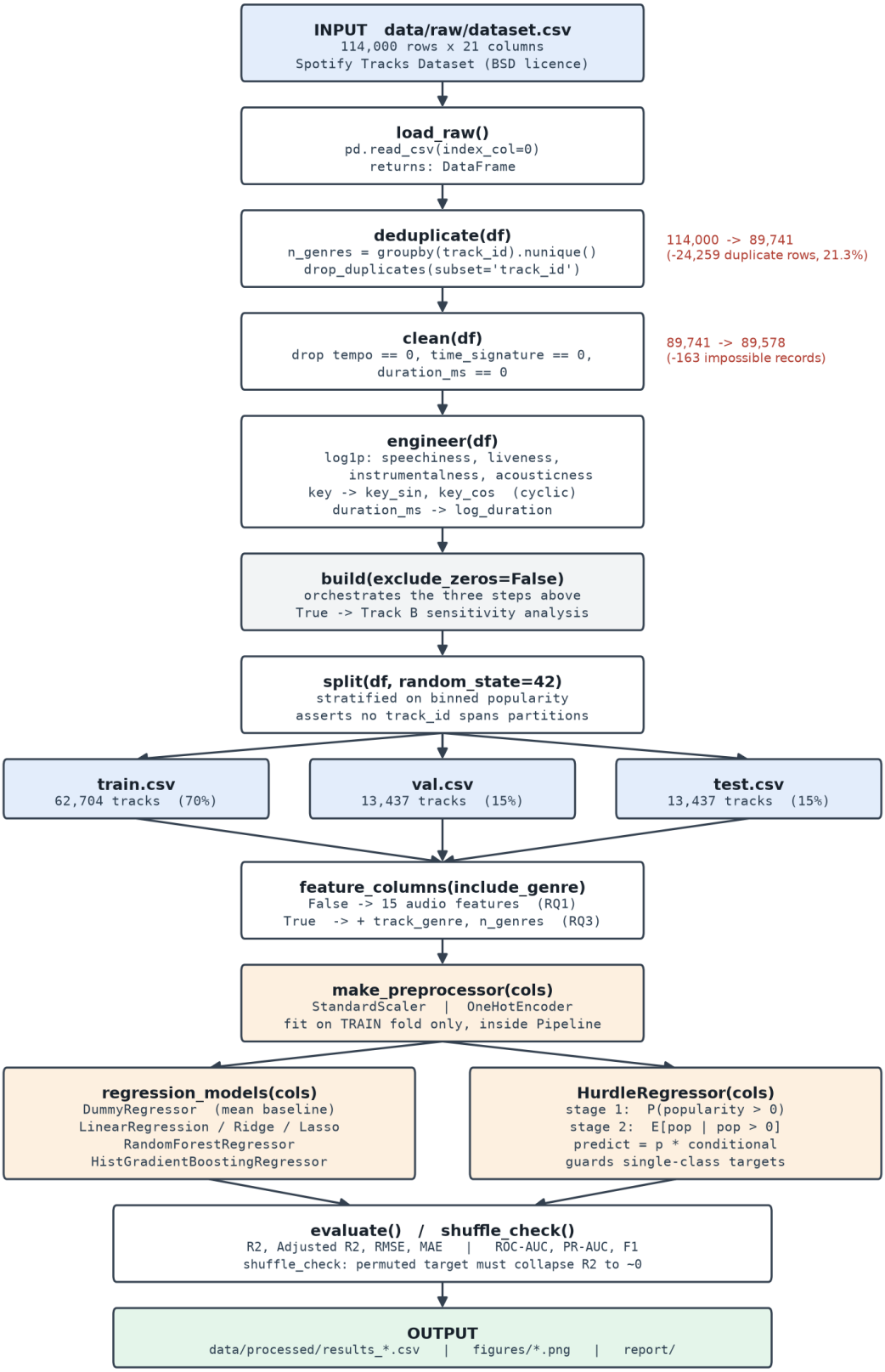


Figure 1 - System flowchart. Inputs, functions, intermediate row counts, the three disjoint dataset partitions, model families, and outputs.

Figure 2 — Module and Class Diagram (UML style)

Members extracted from source; open arrows denote dependency (import / uses)

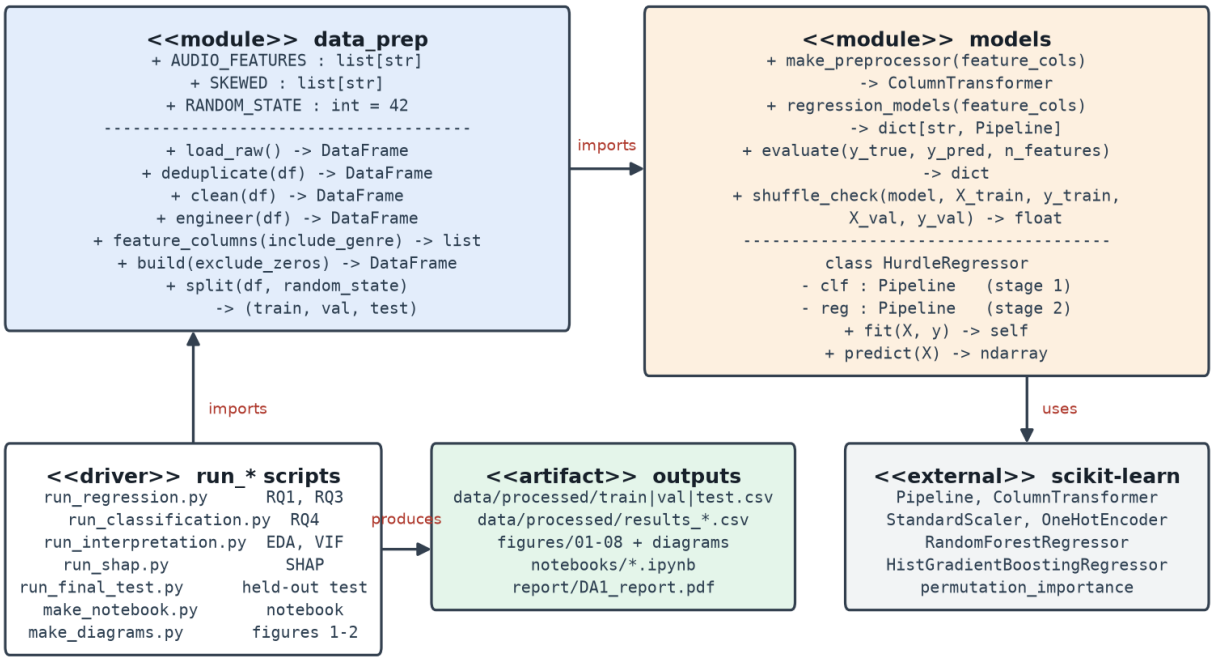


Figure 2 - Module and class diagram. Module members with signatures and return types, the HurdleRegressor class, and dependency direction.

8. Basic Analysis Which Can Be Done

The following analyses are supported by the data. Preliminary results from an initial run are included to demonstrate feasibility; the full study will extend and formalise them.

8.1 Structural and data-quality analysis

- Duplicate audit — quantifying repeated tracks across genres and the leakage they would cause (established: 21.3% of rows).
- Target distribution analysis — characterising the zero-inflated mass and testing whether it is randomly distributed or clustered.
- Missing-value and impossible-value detection.

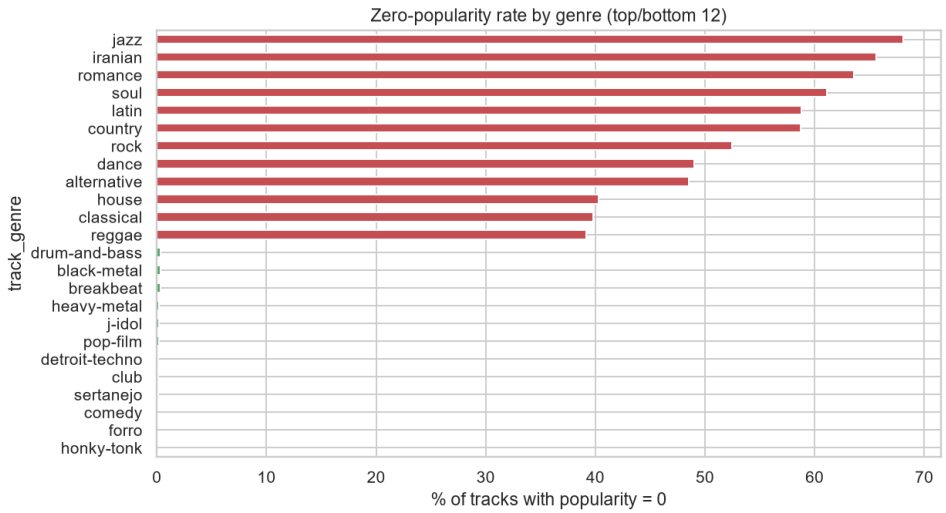


Figure 3 — Zero-popularity rate by genre. The zeros are strongly clustered (jazz 68.1%, Iranian 65.6%) rather than randomly distributed, which indicates regional availability gaps at scrape time rather than unpopular music, and motivates the two-stage hurdle model.

8.2 Correlation and multicollinearity analysis

Pairwise correlation between all audio features and the target, plus formal VIF testing. Preliminary finding: **no audio feature correlates with popularity above |r| = 0.10** (strongest: instrumentalness at –0.095), while energy, loudness and acousticness are strongly inter-correlated. This establishes at the outset that any linear model will explain very little, and that the relationship must be sought in non-linear structure.

8.3 Predictive modelling and comparison

Preliminary held-out test results (evaluated once, after model selection):

Feature set	Model	R^2	RMSE	MAE
Audio only	Mean baseline	0.000	20.64	17.31
Audio only	Linear Regression	0.034	20.28	16.72
Audio only	**Random Forest**	**0.255**	17.81	13.97
Audio + genre	Linear Regression	0.324	16.97	12.16
Audio + genre	**Random Forest**	**0.483**	14.84	10.20

Classification framing (threshold: 75th percentile of non-zero popularity; positive rate 23.7%):

Feature set	Model	ROC-AUC	PR-AUC	PR baseline
Audio only	Random Forest	0.730	0.441	0.237
Audio + genre	Random Forest	**0.870**	**0.679**	0.237

8.4 Feature importance and interpretation

Permutation importance and SHAP analysis to answer RQ2. Preliminary finding: **genre is roughly thirteen times more important than the strongest audio feature** (permutation importance 0.674 versus 0.052). Within the audio features alone, the two methods agree closely on the ranking — instrumentalness, acousticness, valence and duration lead, while key and mode are effectively inert.

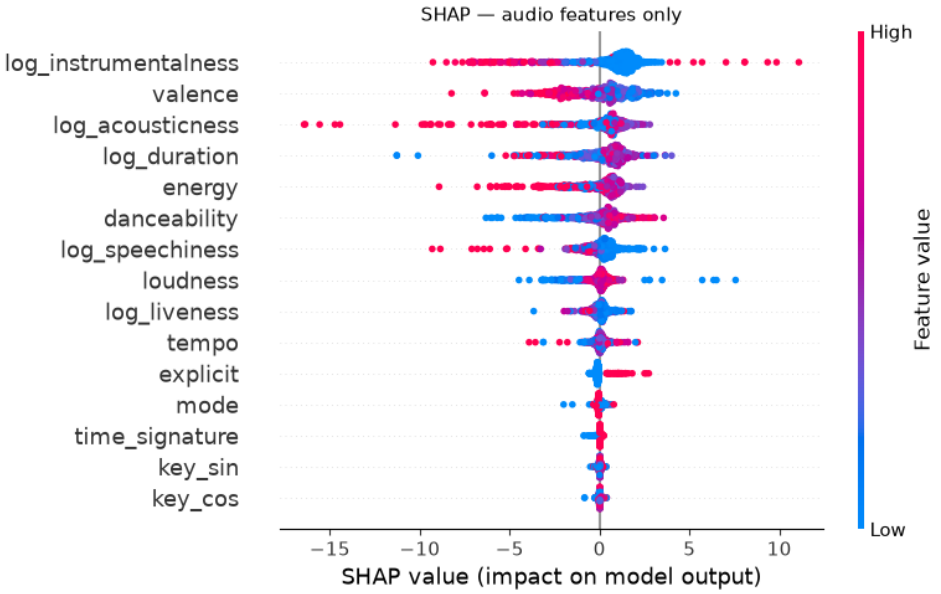


Figure 4 — SHAP summary for the audio-only model. Instrumentalness and acousticness push predicted popularity downward; musical key contributes essentially nothing.

8.5 Validity and robustness analysis

- **Leakage verification** — assertions that no track identifier spans partitions.
- **Shuffled-target control** — preliminary result: R^2 = –0.0016, confirming no leakage path.
- **Sensitivity analysis** — the full study repeated with zero-popularity tracks excluded, testing how far conclusions depend on that decision.
- **Threshold sensitivity** — the classification cut repeated at the 90th percentile.
- **Era confounding** — recording loudness has risen over decades and newer releases score higher through recency bias, so audio features may partly proxy for release date. The primary dataset has no year column, so this will be tested using the secondary dataset described in Section 4.3.

8.6 Expected deliverables

- A reproducible analysis pipeline with leakage controls built into its structure.
- An executed Jupyter notebook presenting the analysis end to end.
- A quantified answer to each of RQ1–RQ4, with an explicit statement of limitations.